

Chapter 05 · SQL Databases

Sub-Chapter 07 — Stored Procedures & Triggers

07 / 10

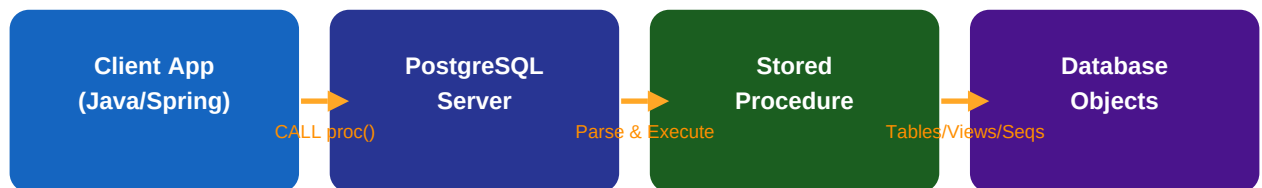
Scope	Stored procedures, functions, triggers, Spring integration
Database	PostgreSQL with PL/pgSQL
Level	Intermediate → Advanced
Outcome	Write, call, and manage server-side DB routines from Spring

1 What Are Stored Procedures & Triggers?

Server-side SQL routines and automatic triggers

A **stored procedure** is a named block of SQL (plus procedural logic) that lives inside the database and can be called by name. A **trigger** is a procedure that fires *automatically* when a specific DML event occurs on a table. Both run **inside the database engine** — close to the data — which means less network round-trips and access to internal row state unavailable to application code.

Stored Procedure Execution Flow



Procedure is compiled once, cached, reused on every CALL

Why use them?

Encapsulate business rules — Logic enforced at DB level regardless of which app calls it.

Reduce network traffic — One CALL replaces dozens of SQL round-trips.

Security boundary — Grant EXECUTE on proc, not SELECT/UPDATE on tables.

Audit & compliance — Triggers automatically log every change without app cooperation.

2 Stored Procedures

PL/pgSQL basics, parameters, error handling

2.1 Creating a Simple Procedure (PostgreSQL)

PostgreSQL uses **PL/pgSQL** (Procedural Language/PostgreSQL). The syntax looks like a mini-program inside SQL:

```
-- Create a procedure to credit a bank account CREATE OR REPLACE PROCEDURE credit_account(  
p_account_id INT, p_amount NUMERIC ) LANGUAGE plpgsql AS $$ DECLARE v_balance NUMERIC; BEGIN --  
Read current balance SELECT balance INTO v_balance FROM accounts WHERE id = p_account_id FOR  
UPDATE; -- Lock the row IF v_balance IS NULL THEN RAISE EXCEPTION 'Account % not found',  
p_account_id; END IF; -- Apply the credit UPDATE accounts SET balance = balance + p_amount,  
updated_at = NOW() WHERE id = p_account_id; -- Insert audit log inside same transaction INSERT INTO  
account_audit (account_id, delta, action, ts) VALUES (p_account_id, p_amount, 'CREDIT', NOW());  
COMMIT; -- explicit commit inside procedure END; $$; -- Calling it: CALL credit_account(42,  
500.00);
```

Key points: **DECLARE** block for variables, **RAISE EXCEPTION** for error handling, **FOR UPDATE** to lock the row preventing concurrent modification, and **COMMIT** inside the procedure (only procedures can commit, not functions).

2.2 IN / OUT / INOUT Parameters

```
CREATE OR REPLACE PROCEDURE transfer_funds( IN p_from_id INT, IN p_to_id INT, IN p_amount NUMERIC,
OUT p_status TEXT -- OUT param: caller reads this ) LANGUAGE plpgsql AS $$ BEGIN UPDATE accounts
SET balance = balance - p_amount WHERE id = p_from_id; UPDATE accounts SET balance = balance +
p_amount WHERE id = p_to_id; p_status := 'SUCCESS'; EXCEPTION WHEN OTHERS THEN p_status := 'ERROR:
' || SQLERRM; ROLLBACK; END; $$; -- Call and read OUT param: DO $$ DECLARE v_status TEXT; BEGIN
CALL transfer_funds(1, 2, 100.00, v_status); RAISE NOTICE 'Result: %', v_status; END; $$;
```

3 Functions

Scalar, table-returning, and aggregate functions

Unlike procedures, **functions** return a value and can appear in SELECT expressions. They **cannot** commit/rollback transactions. Use functions for pure computation and procedures for business workflows.

3.1 Scalar Function

```
-- Returns the age in full years CREATE OR REPLACE FUNCTION fn_age_years(p_dob DATE) RETURNS INT
LANGUAGE sql IMMUTABLE STRICT AS $$ SELECT DATE_PART('year', AGE(CURRENT_DATE, p_dob))::INT; $$;
-- Use in a query – works anywhere an expression is valid SELECT name, fn_age_years(date_of_birth)
AS age FROM customers WHERE fn_age_years(date_of_birth) >= 18;
```

IMMUTABLE tells the planner the function always returns the same result for the same input — the planner can evaluate it once and cache the result.

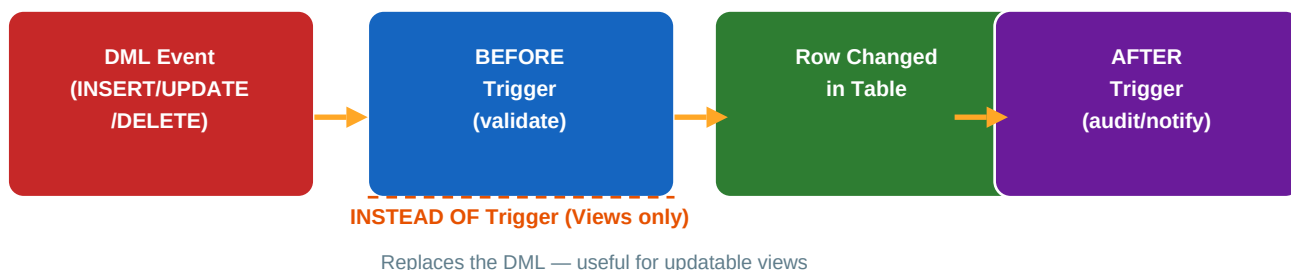
3.2 Table-Returning Function (set-returning)

```
-- Returns a result-set like a virtual table CREATE OR REPLACE FUNCTION
fn_orders_by_customer(p_customer_id INT) RETURNS TABLE(order_id INT, total NUMERIC, status TEXT,
order_date DATE) LANGUAGE plpgsql STABLE AS $$ BEGIN RETURN QUERY SELECT o.id, o.total, o.status,
o.created_at::DATE FROM orders o WHERE o.customer_id = p_customer_id ORDER BY o.created_at DESC;
END; $$; -- Usage: SELECT * FROM fn_orders_by_customer(101); -- Can even JOIN with it: SELECT
c.name, o.* FROM customers c JOIN fn_orders_by_customer(c.id) o ON TRUE WHERE c.region = 'APAC';
```

4 Triggers

BEFORE, AFTER, INSTEAD OF — automatic row-level logic

Trigger Execution Timeline



A trigger has two parts: a **trigger function** (the PL/pgSQL code) and a **trigger definition** (WHEN it fires). The trigger function has access to **NEW** (the new row) and **OLD** (the old row before change).

4.1 AFTER INSERT — Audit Trigger

```
-- Step 1: create the trigger function (returns TRIGGER) CREATE OR REPLACE FUNCTION
trg_audit_orders() RETURNS TRIGGER LANGUAGE plpgsql AS $$ BEGIN INSERT INTO order_audit_log (
order_id, action, changed_by, old_total, new_total, changed_at ) VALUES ( NEW.id, TG_OP, --
'INSERT', 'UPDATE', or 'DELETE' CURRENT_USER, CASE WHEN TG_OP = 'DELETE' THEN OLD.total ELSE NULL
END, CASE WHEN TG_OP != 'DELETE' THEN NEW.total ELSE NULL END, NOW() ); RETURN NEW; -- must return
NEW (or OLD for DELETE) END; $$; -- Step 2: attach the trigger to the table CREATE OR REPLACE
TRIGGER trg_orders_audit AFTER INSERT OR UPDATE OR DELETE ON orders FOR EACH ROW -- row-level (vs.
FOR EACH STATEMENT) EXECUTE FUNCTION trg_audit_orders();
```

TG_OP tells the function what operation fired it. **FOR EACH ROW** fires once per modified row — **FOR EACH STATEMENT** fires once per SQL statement regardless of rows affected.

4.2 BEFORE UPDATE — Validation Trigger

```
CREATE OR REPLACE FUNCTION trg_validate_order_total() RETURNS TRIGGER LANGUAGE plpgsql AS $$ BEGIN
-- Prevent negative totals IF NEW.total < 0 THEN RAISE EXCEPTION 'Order total cannot be negative:
%', NEW.total; END IF; -- Automatically set updated_at NEW.updated_at := NOW(); -- Prevent status
regression (shipped → pending is illegal) IF OLD.status = 'SHIPPED' AND NEW.status = 'PENDING'
THEN RAISE EXCEPTION 'Cannot revert shipped order to pending'; END IF; RETURN NEW; -- returning
modified NEW applies the changes END; $$; CREATE OR REPLACE TRIGGER trg_order_validation BEFORE
UPDATE ON orders FOR EACH ROW EXECUTE FUNCTION trg_validate_order_total();
```

5

Spring & JPA Integration

@Procedure, SimpleJdbcCall, @Query with functions

Spring provides three ways to call stored procedures: **@Procedure** annotations on repository methods, **SimpleJdbcCall** for full control over IN/OUT parameters, and **EntityManager.createStoredProcedureQuery()** for JPA purists.

5.1 @Procedure Annotation (Spring Data JPA)

```
// Entity @Entity @Table(name = "accounts") public class Account { @Id @GeneratedValue private
Long id; private String owner; private BigDecimal balance; private LocalDateTime updatedAt; } //
Repository public interface AccountRepository extends JpaRepository { // Calls PostgreSQL
procedure: credit_account(p_account_id, p_amount) @Procedure(procedureName = "credit_account")
void creditAccount(@Param("p_account_id") Long accountId, @Param("p_amount") BigDecimal amount);
// Procedure with OUT param – return type matches OUT @Procedure(procedureName = "transfer_funds")
String transferFunds(@Param("p_from_id") Long fromId, @Param("p_to_id") Long toId,
@Param("p_amount") BigDecimal amount); } // Service @Service @Transactional public class
AccountService { private final AccountRepository repo; public void credit(Long id, BigDecimal
amount) { repo.creditAccount(id, amount); // single CALL over the wire } public String
transfer(Long from, Long to, BigDecimal amount) { return repo.transferFunds(from, to, amount); } }
```

Spring wraps the CALL in a transaction automatically if the service method is annotated with **@Transactional**.

5.2 SimpleJdbcCall — Full Control

```
@Service public class ReportService { private final SimpleJdbcCall generateReportCall; public
ReportService(DataSource ds) { // Wire up the procedure with explicit param declarations
this.generateReportCall = new SimpleJdbcCall(ds).withProcedureName("generate_monthly_report")
.declareParameters( new SqlParameter("p_year", Types.INTEGER), new SqlParameter("p_month",
Types.INTEGER), new SqlParameter("p_row_count", Types.INTEGER), new SqlParameter("p_status",
Types.VARCHAR) ); } public Map generateReport(int year, int month) { MapSqlParameterSource params
= new MapSqlParameterSource().addValue("p_year", year).addValue("p_month", month); Map result =
generateReportCall.execute(params); int rowCount = (int) result.get("p_row_count"); String status
= (String) result.get("p_status"); log.info("Report generated: {} rows, status={}", rowCount,
status); return result; } }
```

SimpleJdbcCall handles JDBC CallableStatement boilerplate — registering OUT parameters, executing the call, and mapping results.

5.3 Calling Functions via @Query

```
public interface OrderRepository extends JpaRepository { // Call a table-returning function with
    native SQL @Query(value = "SELECT * FROM fn_orders_by_customer(:customerId)", nativeQuery = true)
    List findOrdersByCustomer(@Param("customerId") Long customerId); // Call a scalar function inline
    @Query(value = "SELECT fn_age_years(CAST(:dob AS date))", nativeQuery = true) Integer
    getCustomerAge(@Param("dob") String dob); }
```

6

Procedure vs Function vs View

Decision guide and comparison matrix

Stored Procedure vs Function vs View

Feature	Stored Procedure	Function	View
Returns value	Optional (OUT param)	Required (RETURN)	Always (result set)
Side effects	Yes (DML allowed)	No (pure)	No
CALL syntax	CALL proc()	SELECT func()	SELECT * FROM v
Use in SELECT	No	Yes	Yes
Transaction	Can COMMIT/ROLLBACK	Cannot	N/A
Spring mapping	@Procedure	@Query func()	Repository query

When to choose each:

Stored Procedure — Multi-step business workflow (transfer funds, place order), batch jobs, anything needing COMMIT/ROLLBACK inside the routine.

Function — Pure computation reused in SELECT queries (calculate age, format name, apply discount formula). No side effects.

Trigger — Cross-cutting concerns: audit logging, enforcing complex constraints, automatically populating derived columns, sending notifications.

View — Reusable read-only query with business-meaningful column names; simplify complex JOINS for reporting.

Avoid when — Procedure/trigger logic creates a hidden layer that's hard to test in unit tests. Prefer moving complex orchestration to the service layer if testability is a priority.

7

Advanced Patterns

Generic audit, upsert with RETURNING, pg_cron scheduling

7.1 Trigger-Based Audit Table (full history)

```
-- Generic audit table works for any table via JSONB CREATE TABLE audit_log ( id BIGSERIAL PRIMARY
KEY, table_name TEXT NOT NULL, operation TEXT NOT NULL, -- INSERT/UPDATE/DELETE row_id BIGINT,
old_data JSONB, -- full old row new_data JSONB, -- full new row changed_by TEXT DEFAULT
CURRENT_USER, changed_at TIMESTAMPTZ DEFAULT NOW() ); CREATE OR REPLACE FUNCTION
trg_generic_audit() RETURNS TRIGGER LANGUAGE plpgsql AS $$ BEGIN INSERT INTO audit_log
```

```
(table_name, operation, row_id, old_data, new_data) VALUES ( TG_TABLE_NAME, TG_OP,
COALESCE(NEW.id, OLD.id), CASE WHEN TG_OP != 'INSERT' THEN to_jsonb(OLD) END, CASE WHEN TG_OP !=
'DELETE' THEN to_jsonb(NEW) END ); RETURN COALESCE(NEW, OLD); END; $$; -- Attach to any table with
one line: CREATE TRIGGER audit_orders AFTER INSERT OR UPDATE OR DELETE ON orders FOR EACH ROW
EXECUTE FUNCTION trg_generic_audit();
```

to_jsonb(row) serialises the entire row to JSONB — future column additions are automatically captured without changing the trigger.

7.2 Upsert with RETURNING in a Procedure

```
CREATE OR REPLACE PROCEDURE upsert_product( p_sku TEXT, p_name TEXT, p_price NUMERIC, OUT p_id INT
) LANGUAGE plpgsql AS $$ BEGIN INSERT INTO products (sku, name, price, updated_at) VALUES (p_sku,
p_name, p_price, NOW()) ON CONFLICT (sku) DO UPDATE SET name = EXCLUDED.name, price =
EXCLUDED.price, updated_at = NOW() RETURNING id INTO p_id; COMMIT; END; $$;
```

7.3 Scheduled Procedures with pg_cron

```
-- Install pg_cron extension (once per database) CREATE EXTENSION IF NOT EXISTS pg_cron; -- Run
archive_old_orders() every night at 2 AM UTC SELECT cron.schedule( 'nightly-archive', '0 2 * * *',
$$CALL archive_old_orders()$$ ); -- List scheduled jobs SELECT * FROM cron.job; -- Remove a job
SELECT cron.unschedule('nightly-archive');
```

pg_cron runs inside PostgreSQL itself — no external cron daemon needed. Spring Scheduler can complement this for app-level orchestration while leaving heavy data operations to pg_cron.

8

Quick-Reference Cheatsheet

One-liner reference for all stored routine commands

Task	PostgreSQL SQL	Notes
Create procedure	CREATE OR REPLACE PROCEDURE name(...) LANGUAGE plpgsql AS \$\$\$...\$\$\$	Use CALL to invoke
Create function	CREATE OR REPLACE FUNCTION name(...) RETURNS type LANGUAGE sql AS \$\$\$\$	Use SELECT to invoke
Call procedure	CALL proc_name(arg1, arg2)	Works in psql, JDBC, Spring
BEFORE trigger	CREATE TRIGGER t BEFORE INSERT ON tbl FOR EACH ROW EXECUTE FUNCTION f()	Can modify NEW
AFTER trigger	CREATE TRIGGER t AFTER UPDATE ON tbl FOR EACH ROW EXECUTE FUNCTION f()	Cannot modify NEW
Drop trigger	DROP TRIGGER tname ON table_name	
List procedures	\df (psql) or pg_proc system table	
Spring @Procedure	@Procedure(procedureName="pro c") RetType method(@Param...)	In JPA Repository

SimpleJdbcCall	<code>new SimpleJdbcCall(ds).withProcedureName("proc").execute(params)</code>	Best for OUT params
Disable trigger	<code>ALTER TABLE tbl DISABLE TRIGGER trig_name</code>	Re-enable: <code>ENABLE TRIGGER</code>